

C++ cost US\$ 75, which was the price of the magnetic tape on which it was shipped (source and binary); organised distribution over the Internet was still in the future, back then. Soon, AT&T gave the specifications of C and C++ to the ISO so everyone could use them, and I personally helped other organisations with getting C++ compilers written.

Nowadays, there are, of course, open source and proprietary C++ implementations. To ensure the wide reach of my work, I deliberately (with the agreement of AT&T) refrained from patenting anything related to C++.

### Q Do you see any programming languages today that can replace C++? Or can we call it irreplaceable?

I do not see a current language that could replace C++ across its range of uses. Its combination of hardware access and zero-overhead abstraction is still unique. However, nothing lasts forever.

Eventually, a current language will acquire sufficient facilities or a new language will come along. C++ has certainly been at the top of the game for almost 30 years. That is not bad, especially given that C++ never had a powerful owner or a marketing budget.

### Q What are the major features that make C++ an easier option compared to C?

C++ offers you a better type system, classes with constructors and destructors, overloading, native support for object-oriented programming, support for generic programming as well as compile-time programming.

### Q Why do aspiring developers need to learn C++ to survive in the growing world of computers?

I do not know if they need to learn C++, but they should want to. It is one of the most widely used languages and among the most flexible. It is also one of the languages that delivers the best performance, is very popular and allows you direct access to hardware resources.

Further, C++ is one of the few languages that allow you to use a wide range of fundamental programming techniques. It also allows you to work in most industries.

### Q There are different compilers available for C++. Which one is the best, in your view, and why?

I do not have a favourite compiler for C++. I use several. The major C++ compilers are all good. They have good standard conformance, generate good code and have good supporting toolsets and fundamental libraries. You can choose a C++ compiler based on specific needs such as the ability to use specific hardware or specific programming techniques, specific environments, portability or a certain toolset like GDB or Visual Studio.

“  
Today's C++ is so  
much more readable  
than older C++ or C.  
”

### Q Unlike Python and Java, C++ has not yet become the perfect choice for embedded engineers. Do you think the focus should now also be on connected devices?

Python and Java are not perfect choices for embedded systems either. C++ is critical when you need to squeeze performance or energy efficiency (say, enhancing battery life) out of a gadget, but no one language is the best for everything and everybody. There is a lot of C++ embedded system code. It is worth remembering that ‘embedded systems’ include a vast range of things — from coffee machines to jet plane flight controls, from fuel injectors to stereo amplifiers, and from lithium ion battery controllers to self-driving cars. There is a corresponding range of needs for programming techniques and tools.

### Q How important, do you think, is readability in a programming language?

Readability is essential. If you cannot read the code, you cannot maintain it and cannot discuss or argue about how correct it is. Today's C++ is so much more readable than older C++ or C.

I am working on an ambitious project to define what modern C++ code -- using C++11, C++14, C++17 and beyond — should look like. It is called the Core Guidelines, and is an open source project with editors from Morgan Stanley, Microsoft, Red Hat and Facebook.

We are aiming for completely type-safe and resource-safe code, without limitations on expressibility or performance. Code conforming to the Core Guidelines is far more regular and readable than most current code. Moreover, we are working on tools for the automatic enforcement of these guidelines.

### Q Do you see any big difference between computer systems and embedded devices that are designed for the Internet of Things (IoT) ecosystem?

There are differences such as application binary interfaces (ABIs) and security concerns, but the fundamental programming needs and constraints are the same, and I think they favour C++.

### Q How do you plan the revisions of C++ standards?

We just moved C++17 out of a national vote. It will be the ISO C++ standard later this year. C++20 will be the standard after that in 2020. For C++20, we will code for concepts, modules and possibly contracts and co-routines. It could be an exciting major revision and change the way we program, but all of this will depend on the standards committee's willingness to accept change.

Concepts are shipping in GCC, while modules are shipping in Microsoft and co-routines are available in Microsoft and Clang. More implementations are